UMassAmherst | Manning College of Information & Computer Sciences

Programming Methodology

Lab 9: Midterm 2 Review

Wednesday, April 8th, 2026

Reminders

- **Midterm 2 is tonight, 7-9pm!**
  - **Thompson 102: Lecture section 2 (Prof. Minea)**
  - **Thompson 104: Lecture section 1 (Prof. Perretta)**
  - Advice:
    - Write legibly (reduces risk of misgrading)
    - Write the types of function parameters and return values
    - The order of the questions doesn't mean anything, feel free to do them in whatever order is best for you
- **Homework 8 questionnaires are due tonight at 11:59pm!**
  - CATME
  - Google Form

Reminders

- **Labs are due at 4pm** and intended to be completed in class
  - **Download and submit lab code at the beginning of lab**
    - Ensures that at least a blank submission is recorded
  - **Submit your code again at the end of lab**
- Please **arrive to lab on time** and **stay until the end**
  - It is disruptive to the class when people arrive late
  - This lab is built into your schedule and the explained solutions to lab exercises will bolster your understanding of course materials

Today's Goals

- Review midterm 2 material
  - Property-Based Testing
  - Iterators
  - Generators
  - Extending Libraries
  - Observer, Singleton, and Model-View-Controller Design Patterns

Topic Poll

Which topics do you most want to review?

- Property-Based Testing
- Iterators
- Generators
- Extending Libraries
- Observer, Singleton, and Model-View-Controller Design Patterns

Exercise 1: Property-Based Testing

Write property-based tests for the function

```
partition(arr: number[], p: number)
```

with the following specification:

- The array elements and **p** are assumed to be integers between 0 and **`arr.length-1`**.
- The function rearranges the array in place, so that a number with value > **p** will not appear before a number with value <= **p**, and a number with value < **p** will not appear after a number with value >= **p**.

Try to write a complete set of tests, adding comments to clearly indicate the purpose of each test.

Exercise 1: Solution

Tests:

1. Check that the original and resulting array have the same length.
2. Check that they contain the same elements with the same frequencies. Together, this can be done by making a copy before calling **`partition`** and comparing.
3. Check that the result has elements less than **p**, then elements equal to **p**, then elements greater than **p** (any of those may be missing).

Exercise 1: Solution

Tests:

1. Check that the original and resulting array have the same length.

```
function checkSame(a: number[], b: number[]): void {
  assert(a.length === b.length);
  // ...
}
```

We will write a helper function **`checkSame`** for the first two tests we have described.

We begin by simply checking if the two arrays have the same length.

Exercise 1: Solution

Tests:

2. Check that they contain the same elements with the same frequencies. Together, this can be done by making a copy before calling partition and comparing.

```
function checkSame(a: number[], b: number[]): void {
  assert(a.length === b.length);
  const isValid = (e: number) => 0 <= e && e < a.length && e % 1 === 0;
  let counts = new Array<number>(a.length).fill(0);
  // ...
}
```

We define a helper function **`isValid`** to determine if a number is an integer between 0 and **a.length-1**, as the problem specifies.

We then create an array **`counts`** to track the number of times each number appears, initializing all counts to 0.

Exercise 1: Solution

Tests:

2. Check that they contain the same elements with the same frequencies. Together, this can be done by making a copy before calling partition and comparing.

```
function checkSame(a: number[], b: number[]): void {
  assert(a.length === b.length);
  const isValid = (e: number) => 0 <= e && e < a.length && e % 1 === 0;
  let counts = new Array<number>(a.length).fill(0);
  b.forEach(e => {
    assert(isValid(e));
    ++counts[e];
  });
  // ...
}
```

We use **`forEach`** to check that each element in **`b`** (result from partition) is valid and to update **`counts`**, tracking the number of times each value appears in **`b`**.

Exercise 1: Solution

Tests:

2. Check that they contain the same elements with the same frequencies. Together, this can be done by making a copy before calling partition and comparing.

```
function checkSame(a: number[], b: number[]): void {
  assert(a.length === b.length);
  const isValid = (e: number) => 0 <= e && e < a.length && e % 1 === 0;
  let counts = new Array<number>(a.length).fill(0);
  b.forEach(e => {
    assert(isValid(e));
    ++counts[e];
  });
  a.forEach(e => assert(--counts[e] >= 0));
}
```

We call **forEach** on **a** (the original array) and ensures that each element does not appear more often in **a** than in **b**.

Exercise 1: Solution

```
function checkSame(a: number[], b: number[]): void {
  assert(a.length === b.length);
  const isValid = (e: number) => 0 <= e && e < a.length && e % 1 === 0;
  let counts = new Array<number>(a.length).fill(0);
  b.forEach(e => {
    assert(isValid(e));
    ++counts[e];
  });
  a.forEach(e => assert(--counts[e] >= 0));
}
```

Is the last line sufficient to check that elements have the same frequencies in both arrays?

Exercise 1: Solution

```
function checkSame(a: number[], b: number[]): void {
  assert(a.length === b.length);
  const isValid = (e: number) => 0 <= e && e < a.length && e % 1 === 0;
  let counts = new Array<number>(a.length).fill(0);
  b.forEach(e => {
    assert(isValid(e));
    ++counts[e];
  });
  a.forEach(e => assert(--counts[e] >= 0));
}
```

Is the last line sufficient to check that elements have the same frequencies in both arrays?

Yes. The last check ensures that each element in **a** does not appear more often than it does in **b**. Now, if there was an element of **b** that appears more often in **b** than in **a**, then **counts** should have at least one non-zero count after the last line.

However, this would require **b** to be longer than **a**, which we know is not true.

Exercise 1: Solution

Tests:

3. Check that the result has elements less than **p**, then elements equal to **p**, then elements greater than **p** (any of those may be missing).

```
function checkPivot(a: number[], p: number): void {
  let i = a.length; if (i === 0) { return; }
  while (i > 0 && a[--i] > p) {} //all > p
  while (i >= 0 && a[i] === p) { --i; } //all = p
  while (i >= 0) { assert(a[i] < p); --i; }
}
```

For the third test, we can iterate through the resulting array from right to left. We use while loops to first iterate through all elements greater than **p**, then all elements equal to **p**, and lastly assert that all remaining elements are less than **p**.

Exercise 1: Solution

Tests:

1. Check that the original and resulting array have the same length
2. Check that they contain the same elements with the same frequencies. Together, this can be done by making a copy before partition and comparing:
3. Check that the result has elements less than **p**, then elements equal to **p**, then elements greater than **p** (any of those may be missing).

```
function checkSame(a: number[], b: number[]): void {
  assert(a.length === b.length);
  const isValid = (e: number) =>
    0 <= e && e < a.length && e%1 === 0;
  let f = new Array<number>(a.length).fill(0);
  b.forEach(e => { assert(inRange(e)); ++f[e]; });
  a.forEach(e => assert(--f[e] >= 0));
}

function checkPivot(a: number[], p: number): void {
  let i = a.length; if (i === 0) { return; }
  while (i > 0 && a[--i] > p) {} //all > p
  while (i >= 0 && a[i] === p) { --i; } //all = p
  while (i >= 0) { assert(a[i] < p); --i; }
}
```

You would then use these two functions to test the output of any given partition function to check if it follows the specifications.

Exercise 2: Iterators

Write a function **`makeWindowSums`** that takes an **`Iterable<number>`**, and returns an **`Iterator<number>`**. The returned iterator should return the sum of every 3 consecutive numbers produced by the iterable.

Specifically, suppose the given iterable produces the sequence $(a_1, a_2, \ldots, a_n)$, where **`n`** is the total number of values produced by the iterable. Then, the `i`-th value yielded by the iterator should be equal to $a_i + a_{i+1} + a_{i+2}$. If any of these values go past the end of the sequence $(a_1, a_2, \ldots, a_n)$, your iterator should terminate.

You should design a class **`WindowSumsIterator`** that implements the **`Iterator`** interface, and use this class in your implementation of **`makeWindowSums`**. Do not store the entire iterable as an array.

Exercise 2: Solution

```
class WindowSumsIterator implements Iterator<number> {
    private iterator: Iterator<number>;
    private item1: IteratorResult<number>;
    private item2: IteratorResult<number>;

    // ...
```

We begin by defining three private attributes. The **`iterator`** attribute will allow us to store an iterator for the given iterable, and the item attributes will store the previous two IteratorResults from **`this.iterator`**.

Exercise 2: Solution

```
class WindowSumsIterator implements Iterator<number> {
    private iterator: Iterator<number>;
    private item1: IteratorResult<number>;
    private item2: IteratorResult<number>;

    constructor(iterable: Iterable<number>) {
        this.iterator = iterable[Symbol.iterator]();
        this.item1 = this.iterator.next();
        this.item2 = this.iterator.next();
    }

    // ...
```

For the constructor, we take an iterable as input and obtain an iterator for it using the **`[Symbol.iterator]()`** method. We then use **`.next()`** to get the first two items, storing them in their respective attributes.

Exercise 2: Solution

```
class WindowSumsIterator implements Iterator<number> {
    private iterator: Iterator<number>;
    private item1: IteratorResult<number>;
    private item2: IteratorResult<number>;

    next(): IteratorResult<number> {
        const curr = this.iterator.next();
        if (curr.done) {
            return {done: true, value: undefined};
        }
        const sum = this.item1.value + this.item2.value + curr.value;
        // ...
```

We obtain the current **`IteratorResult`** from the iterator. If it indicates that the iterator is done, then our iterator has finished and we return the **`IteratorResult`** indicating this. Otherwise, we calculate the sum of the previous 2 values and the current value.

Exercise 2: Solution

```
class WindowSumsIterator implements Iterator<number> {
    private iterator: Iterator<number>;
    private item1: IteratorResult<number>;
    private item2: IteratorResult<number>;

    next(): IteratorResult<number> {
        const curr = this.iterator.next();
        if (curr.done) {
            return {done: true, value: undefined};
        }
        const sum = this.item1.value + this.item2.value + curr.value;
        this.item1 = this.item2;
        this.item2 = curr;
        return {done: false, value: sum}; }}
```

We set up **`this.item1`** and **`this.item2`** for future calls to **`next()`**. We must store **`curr`** as a previous item, so we move **`this.item2`** into **`this.item1`**. We return an **`IteratorResult`**.

Exercise 2: Solution

```
function makeWindowSums(iterable: Iterable<number>): Iterator<number> {
    return new WindowSumsIterator(iterable);
}
```

Lastly, we use the class we defined in the **`makeWindowSums`** function, returning a new instance of the **`WindowSumsIterator`** class.

Exercise 3: Generators

Write a generator function **`minimaGenerator`** that takes an **`Iterator<number>`** as input and returns a **`Generator<number>`**.

- The generator should yield all the minima in the sequence of numbers returned by the iterator in their original order.
- A number x is a minimum if a number exists before and after it in the sequence, and x is strictly less than both of these two numbers.

For example, if the given iterator yields the sequence (3, 2, 4, 1, 5, 0), then your generator should yield the sequence (2, 1).

Exercise 3: Solution

```
function* minimaGenerator(iterator: Iterator<number>): Generator<number> {
    let prev = iterator.next();
    let curr = iterator.next();
    while (!curr.done) {
        // ...
    }
}
```

We begin by initializing two variables to track the previous and current values from the iterator. We will also need to track a third value (the next value) to determine if **`curr`** is a minima, but this will be done within the while loop.

Exercise 3: Solution

```
function* minimaGenerator(iterator: Iterator<number>): Generator<number> {
    let prev = iterator.next();
    let curr = iterator.next();
    while (!curr.done) {
        let next = iterator.next();
        if (!next.done && curr.value < Math.min(prev.value, next.value))
            yield curr.value;
        // ...
    }
}
```

We obtain the next item from the iterator. If the iterator is not finished yet, then **`next`** will be an **`IteratorResult`** with a value.

After we check that **`next.done`** is false, we check if our **`curr.value`** is a minimum by seeing if it is strictly less than **`prev.value`** and **`next.value`**. We yield the current value if it is a minimum.

Exercise 3: Solution

```
function* minimaGenerator(iterator: Iterator<number>): Generator<number> {
    let prev = iterator.next();
    let curr = iterator.next();
    while (!curr.done) {
        let next = iterator.next();
        if (!next.done && curr.value < Math.min(prev.value, next.value)) {
            yield curr.value;
        }
        prev = curr;
        curr = next;
    }
}
```

Lastly, we update **`prev`** and **`curr`** for the next iteration. **`curr`** moves back and becomes **`prev`**, and **`next`** is stored as **`curr`**.

Exercise 4: Extending Libraries

From a shapes library, you are given a class **`Star`** with the following methods:

- **`constructor(x: number, y: number, r: number)`**
  - **`x`** and **`y`** are the coordinates, **`r`** is the radius
- **`moveTo(x1: number, y1: number): Star`**
  - returns **`this`**
- **`grow(m: number): Star`**
  - returns a new **`Star`** at the same coordinates with the radius multiplied by **`m`**

Implement a new class **`Star2`**, which has the following new methods, in addition to those of **`Star`**:

- **`moveBy(dx, dy): Star2`**
  - returns **`this`**, animates the move by **`dx`** and **`dy`** to new coordinates
- **`shrink(d: number): Star2`**
  - returns a new **`Star2`** at the same coordinates, with radius divided by **`d`**

The new class should not have any properties except for the above methods.

Exercise 4: Solution

```
class Star2 {
    private star: Star;

    constructor(private x: number, private y: number, private r: number) {
        this.star = new Star(x, y, r);
    }

    // ...
}
```

We need to use wrapping because inheriting **`grow`** would return a **`Star`**, not **`Star2`**.

We begin by declaring the wrapped **`star`** property and initializing it in the constructor along with the three numerical properties.

Exercise 4: Solution

```
class Star2 {
    // ...

    grow(m: number) {
        return new Star2(this.x, this.y, this.r * m);
    }

    // ...
}
```

We override **grow** to return a **Star2** instead of **Star**.

Exercise 4: Solution

```
class Star2 {
    // ...

    moveTo(x1: number, y1: number) {
        this.star.moveTo(x1, y1);
        this.x = x1;
        this.y = y1
        return this;
    }

    // ...
}
```

We override **moveTo** to utilize the wrapped instance of **`Star`**.

Exercise 4: Solution

```
class Star2 {
    // ...

    shrink(d: number): Star2 {
        return this.grow(1 / d);
    }

    // ...
}
```

For **`shrink`**, we utilize the existing **`grow`** method to “grow” our star by a factor of **`1 / d`**.

Exercise 4: Solution

```
class Star2 {
    // ...

    moveBy(dx: number, dy: number): Star2 {
        return this.moveTo(this.x + dx, this.y + dy);
    }
}
```

For **moveBy**, we utilize the existing **moveTo** method and add the distances to the current coordinates.

Exercise 4: Solution

```
class Star2 {
    private star: Star;

    constructor(private x: number, private y: number, private r: number) {
        this.star = new Star(x, y, r); }

    grow(m: number) { return new Star2(this.x, this.y, this.r * m); }

    moveTo(x1: number, y1: number) {
        this.star.moveTo(x1, y1);
        this.x = x1; this.y = y1;
        return this; }

    shrink(d: number): Star2 { return this.grow(1 / d); }

    moveBy(dx: number, dy: number): Star2 {
         return this.moveTo(this.x + dx, this.y + dy); }
}
```

Exercise 5: Observers, Singleton, and MVC

Which scenario best captures the core problem the Observer pattern is designed to solve?

A. Ensuring multiple objects can modify the same data concurrently

B. Allowing one object to notify multiple dependent objects automatically when its state changes

C. Reducing the number of classes in a system

D. Enforcing strict control over object creation

Exercise 5: Observers, Singleton, and MVC

Which scenario best captures the core problem the Observer pattern is designed to solve?

A. Ensuring multiple objects can modify the same data concurrently

B. Allowing one object to notify multiple dependent objects automatically when its state changes

C. Reducing the number of classes in a system

D. Enforcing strict control over object creation

Answer: B

Exercise 5: Observers, Singleton, and MVC

Which statement best explains the interaction flow between Subject and Observers?

A. The Subject directly modify the internal state of the Observers

B. The Observers pull data from the Subject when needed

C. The Subject notifies the Observers, and the Observers react by updating themselves

D. Observers continuously request updates in a loop

Exercise 5: Observers, Singleton, and MVC

Which statement best explains the interaction flow between Subject and Observers?

A. The Subject directly modify the internal state of the Observers

B. The Observers pull data from the Subject when needed

C. The Subject notifies the Observers, and the Observers react by updating themselves

D. Observers continuously request updates in a loop

Answer: C

Exercise 5: Observers, Singleton, and MVC

How is the Model–View–Controller pattern commonly related to the Observer pattern?

A. The Controller replaces the need for any notification system

B. The Model often uses Observer-like behavior to notify Views of changes

C. The Model directly modifies the Views without communication rules

D. The View continuously polls the Model for updates

Exercise 5: Observers, Singleton, and MVC

How is the Model–View–Controller pattern commonly related to the Observer pattern?

A. The Controller replaces the need for any notification system

B. The Model often uses Observer-like behavior to notify Views of changes

C. The Model directly modifies the Views without communication rules

D. The View continuously polls the Model for updates

Answer: B

Exercise 5: Observers, Singleton, and MVC

Which part of the pattern is responsible for subscribing and unsubscribing Views to/from the Model?

A. Model

B. View

C. Controller

Exercise 5: Observers, Singleton, and MVC

Which part of the pattern is responsible for subscribing and unsubscribing Views to/from the Model?

A. Model

B. View

C. Controller

Answer: C

Exercise 5: Observers, Singleton, and MVC

Open ended: What are the individual responsibilities of the Model, the View, and the Controller in the Model-View-Controller design pattern?

Exercise 5: Observers, Singleton, and MVC

Open ended: What are the individual responsibilities of the Model, the View, and the Controller in the MVC Design Pattern?

Answer:

- The Model handles data and business logic
- The View displays data
- The Controller handles input and invokes the Model and View

Exercise 5: Observers, Singleton, and MVC

Open ended: What is the main purpose of the Singleton pattern?

Exercise 5: Observers, Singleton, and MVC

Open ended: What is the main purpose of the Singleton pattern?

Answer: To ensure a class has exactly one instance and provide global access to it